In [1]:
```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accu
```

In [2]:
```
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/nursery/

columns = [
    "parents", "has_nurs", "form", "children",
    "housing", "finance", "social", "health", "class"
]

data = pd.read_csv(url, header=None, names=columns)

# Encode categorical values into numbers
encoders = {}

for column in data.columns:
    encoders[column] = LabelEncoder()
    data[column] = encoders[column].fit_transform(data[column])

X = data.drop("class", axis=1)
y = data["class"]

print(data.head())
```

```
   parents  has_nurs  form  children  housing  finance  social  health  cl
ass
0        2         3     0         0        0        0       0       2
2
1        2         3     0         0        0        0       0       1
1
2        2         3     0         0        0        0       0       0
0
3        2         3     0         0        0        0       2       2
2
4        2         3     0         0        0        0       2       1
1
```

In [3]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

In [4]:
```
classifier = DecisionTreeClassifier(
    criterion='gini',
    max_depth=3,
    random_state=42
)
```

In [5]:
```
classifier.fit(X_train, y_train)
```

Out[5]:

▾ DecisionTreeClassifier ⓘ ?

▸ Parameters

▸ Fitted attributes

In [6]:
```
y_pred = classifier.predict(X_test)
```

In [7]:
```
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy of the model:", accuracy)

print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=encoders["class"].classes_
))
```

```
Confusion Matrix:
[[870   0   0   0   0]
 [  0 554   0 319   0]
 [  0   2   0   0   0]
 [  0  95   0 690   0]
 [  0  49   0  13   0]]
Accuracy of the model: 0.8155864197530864

Classification Report:
              precision    recall  f1-score   support

   not_recom       1.00      1.00      1.00       870
    priority       0.79      0.63      0.70       873
   recommend       0.00      0.00      0.00         2
  spec_prior       0.68      0.88      0.76       785
  very_recom       0.00      0.00      0.00        62

    accuracy                           0.82      2592
   macro avg       0.49      0.50      0.49      2592
weighted avg       0.81      0.82      0.80      2592
```

```
/opt/anaconda3/lib/python3.14/site-packages/sklearn/metrics/_classificatio
n.py:1879: UndefinedMetricWarning: Precision is ill-defined and being set
to 0.0 in labels with no predicted samples. Use `zero_division` parameter
to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape
[0])
/opt/anaconda3/lib/python3.14/site-packages/sklearn/metrics/_classificatio
n.py:1879: UndefinedMetricWarning: Precision is ill-defined and being set
to 0.0 in labels with no predicted samples. Use `zero_division` parameter
to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape
[0])
/opt/anaconda3/lib/python3.14/site-packages/sklearn/metrics/_classificatio
n.py:1879: UndefinedMetricWarning: Precision is ill-defined and being set
to 0.0 in labels with no predicted samples. Use `zero_division` parameter
to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape
[0])
```

In [19]:
```
new_sample = pd.DataFrame([{
    "parents": "usual",
    "has_nurs": "proper",
    "form": "complete",
    "children": "3",
    "housing": "convenient",
    "finance": "convenient",
    "social": "nonprob",
    "health": "recommended"
}])


for column in new_sample.columns:
    new_sample[column] = encoders[column].transform(new_sample[column])

predicted_class = classifier.predict(new_sample)

print(
    "\nPrediction for the new sample:",
    encoders["class"].inverse_transform(predicted_class)[0]
)
```

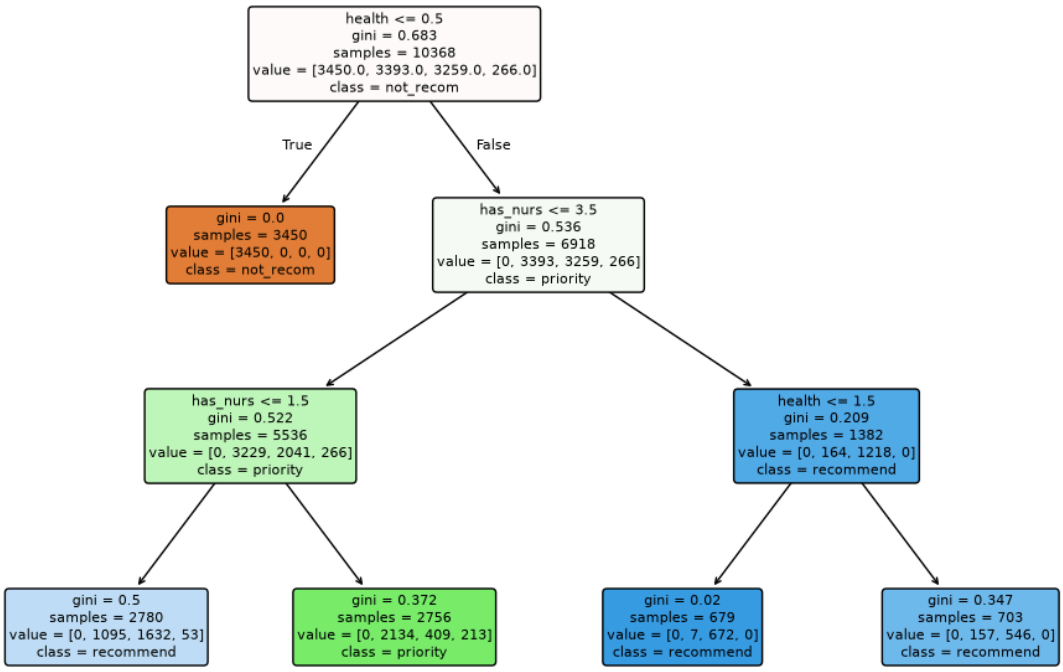
```
Prediction for the new sample: priority
```

In [8]:
```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 8))

plot_tree(
    classifier,
    feature_names=X.columns,
    class_names=encoders["class"].classes_,
    filled=True,
    rounded=True
)

plt.show()
```


health <= 0.5
gini = 0.683
samples = 10368
value = [3450.0, 3393.0, 3259.0, 266.0]
class = not_recom
True
False
gini = 0.0
samples = 3450
value = [3450, 0, 0, 0]
class = not_recom
has_nurs <= 3.5
gini = 0.536
samples = 6918
value = [0, 3393, 3259, 266]
class = priority
has_nurs <= 1.5
gini = 0.522
samples = 5536
value = [0, 3229, 2041, 266]
class = priority
health <= 1.5
gini = 0.209
samples = 1382
value = [0, 164, 1218, 0]
class = recommend
gini = 0.5
samples = 2780
value = [0, 1095, 1632, 53]
class = recommend
gini = 0.372
samples = 2756
value = [0, 2134, 409, 213]
class = priority
gini = 0.02
samples = 679
value = [0, 7, 672, 0]
class = recommend
gini = 0.347
samples = 703
value = [0, 157, 546, 0]
class = recommend

In [ ]: